Industrial Internet of Things
NodeJS, Server, MQTT, Broker

Dr. Ir. Eko Mursito Budi, M.T.

Faqihza Mukhlish, S.T., M.T., Ph.D

S1 Teknik Fisika ITB

2024

NodeJS

What is?

NodeJS

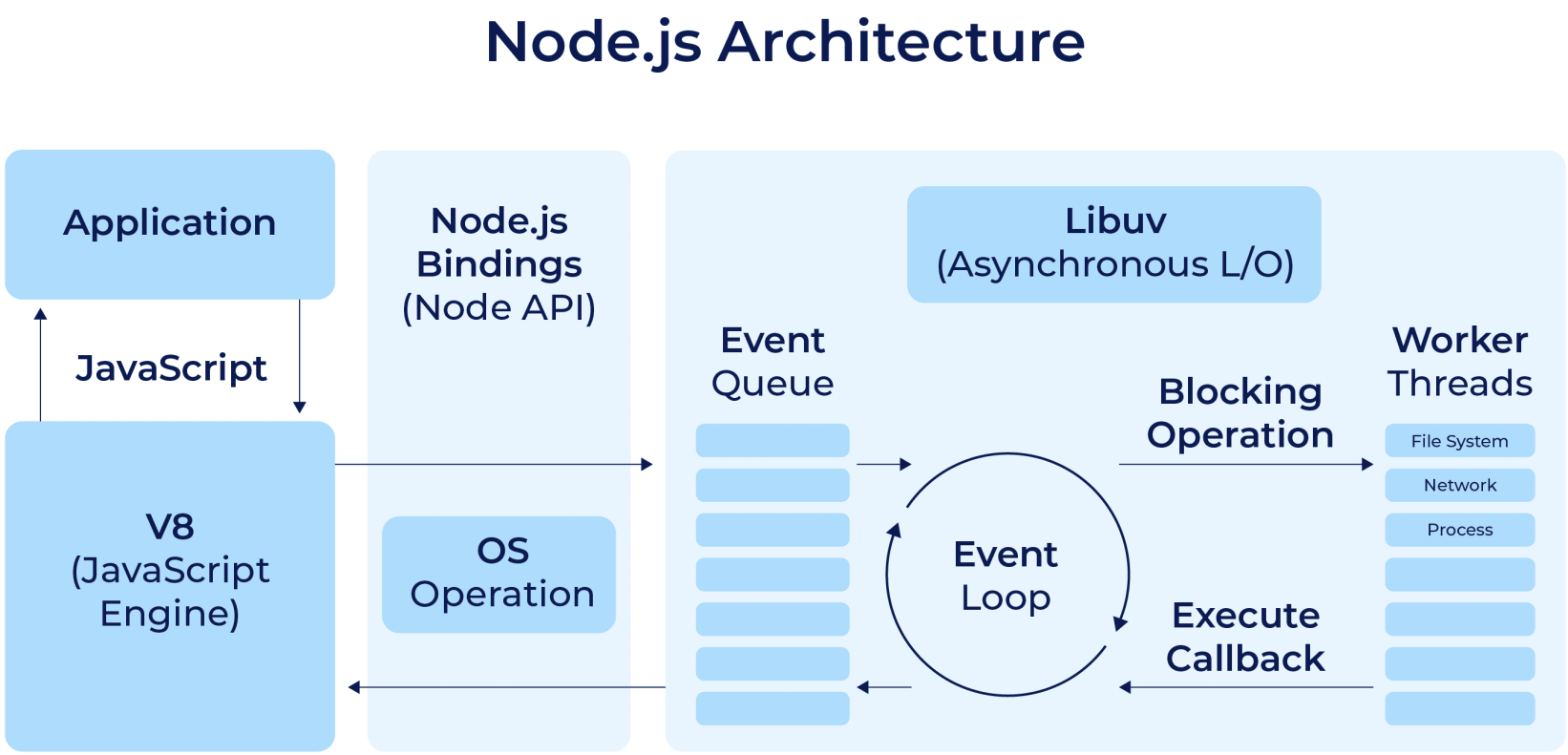
- Node.js is a javascript runtime built on Chrome's V8 Javascript engine
- Node.js uses an event-driven, asynchronous non-blocking I/O model
- Node.js operates on single thread event loop

NodeJS Usage

Top Use Cases of Node.js

- Real-time apps for different online activities
- IoT device apps
- Complex single-page applications (SPAs)
- Real-time collaboration tools
- Real-time chats
- Streaming apps
- Microservices

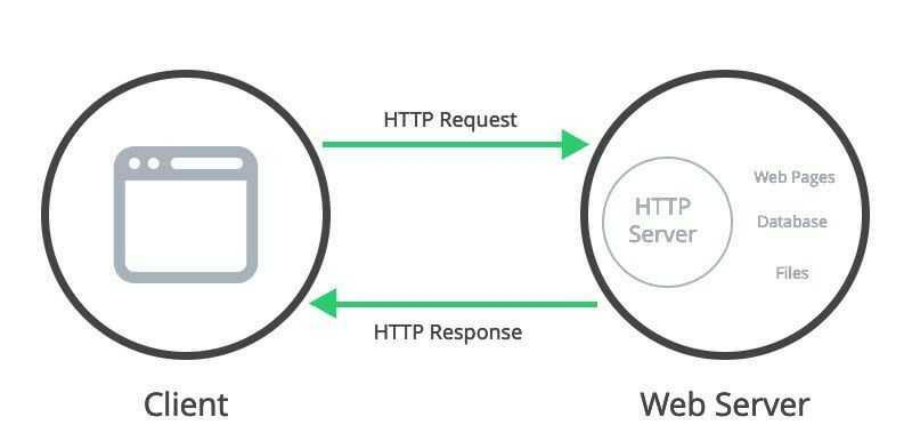
NodeJS Architecture



I/O (Input/Output)

- Short for input/output, **I/O** refers primarily to the program's interaction with the system's disk and network.
  - Examples of I/O operations include reading/writing data from/to a disk,
  - making HTTP requests,
  - and talking to databases.

- They are very slow compared to accessing memory (RAM) or doing work on the CPU.



Synchronous vs Asynchronous

- **Synchronous (or sync)** execution usually refers to code executing in sequence. In sync programming, the program is executed line by line, one line at a time. Each time a function is called, the program execution waits until that function returns before continuing to the next line of code.
- **Asynchronous (or async)** execution refers to execution that doesn't run in the sequence it appears in the code. In async programming the program doesn't wait for the task to complete and can move on to the next task.

Sync vs Async (cont'd)

```
// Synchronous: 1,2,3
alert(1);
alert(2);
alert(3);
```

```
// Asynchronous: 1,3,2
alert(1);
setTimeout(()=>alert(2),0);
alert(3);
```

Blocking vs Non-Blocking

Blocking refers to operations that block further execution until that operation finishes while **non-blocking** refers to code that doesn't block execution. Or as Node.js docs puts it, blocking is when the execution of additional JavaScript in the Node.js process must wait until a non-JavaScript operation completes.

```
// Blocking
const fs = require('fs');
const data = fs.readFileSync('/file.md');

// blocks here until file is read
console.log(data);

moreWork();
// will run after console.log
```

```
// Non-blocking
const fs = require('fs');
fs.readFile('/file.md', (err, data) => {
        if (err) throw err;
        console.log(data);
});

moreWork();
// will run before console.log
```

Callbacks

A **callback** is a function passed as an argument into another function, which can then be invoked (called back) inside the outer function to complete some kind of action at a convenient time. The invocation may be immediate (sync callback) or it might happen at a later time (async callback).

```
// Sync callback
function greetings(callback) {
callback();
}

greetings(() => { console.log('Hi'); }); // anonymous function (arrow) as argument
moreWork();
// will run after console.log

// Async callback
const fs = require('fs’);
// fs.readFile is an async method provided by Node
fs.readFile('/file.md', function callback(err, data) {
          if (err) throw err;
          console.log(data);
});
moreWork(); // will run before console.log
```

Setup NodeJS

See installation guide for more detailed steps

Project Structures

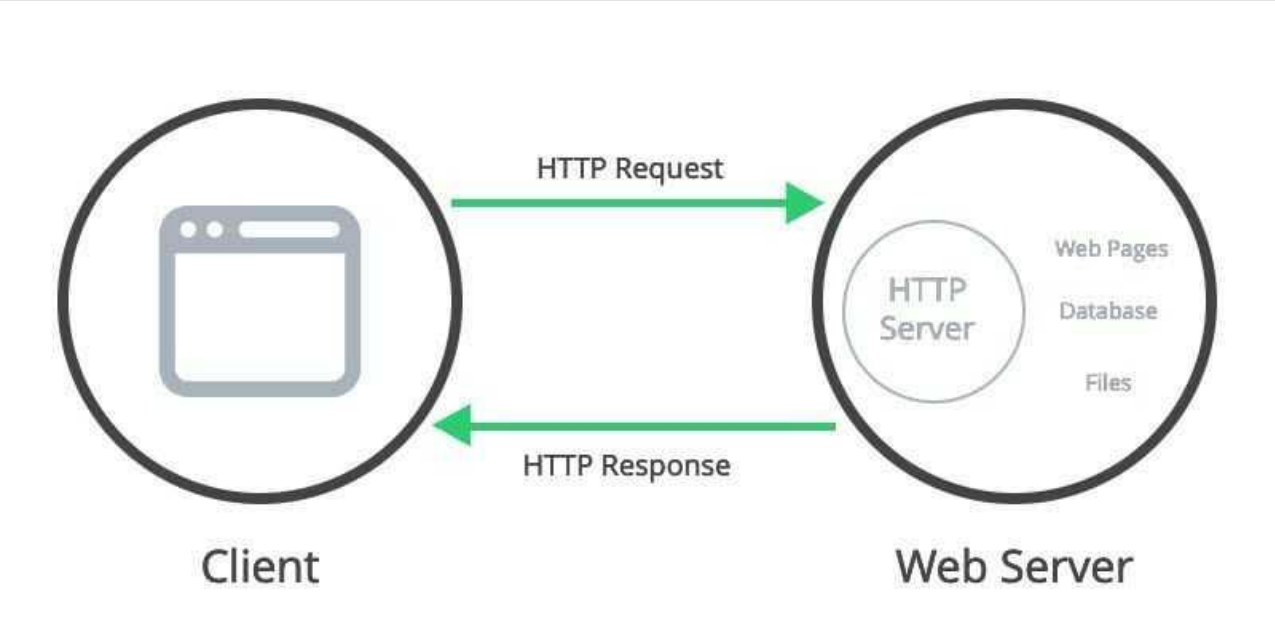
App

- node_modules/
- package.json
- app.js
- *.js (file_service)
- public/
    - folder (css, js, img, dll)
    - index.html
    - *.html (file)

Web Server

Using NodeJS

HTTP Request and Response



Simple Server can be built using HTTP module provided by NodeJS

- Visit lecture's repository on github: code

Code Example (simple server)

app.js

```
const http = require("http");
const host = 'localhost';
const port = 8000;

const requestListener = function(req,res){
    res.writeHead(200);
    res.end("Server live!");
}

const server = http.createServer(requestListener);

server.listen(port,host,() =>{
    console.log(`Server is running on http://${host}:${port}`);
});
```

Visit lecture's repository on github: [code]

Code Example (with HTML)

app.js

```javascript
const http = require("http");
const fs = require("fs");
const host = 'localhost';
const port = 8000;

const requestListener = function(req,res){
    fs.readFile('index.html',function(err,data){
        res.writeHead(200,{'Content-Type':'text/html'});
        res.write(data);
        return res.end();
    });
}

const server = http.createServer(requestListener);

server.listen(port,host,() =>{
    console.log(`Server is running on http://${host}:${port}`);
});
```

index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>NodeJS:01-Hello World</title>
</head>
<body>
    <h1>Hello World!</h1>
    <p>Program pertama menggunakan HTML dan NodeJS</p>
</body>
</html>
```

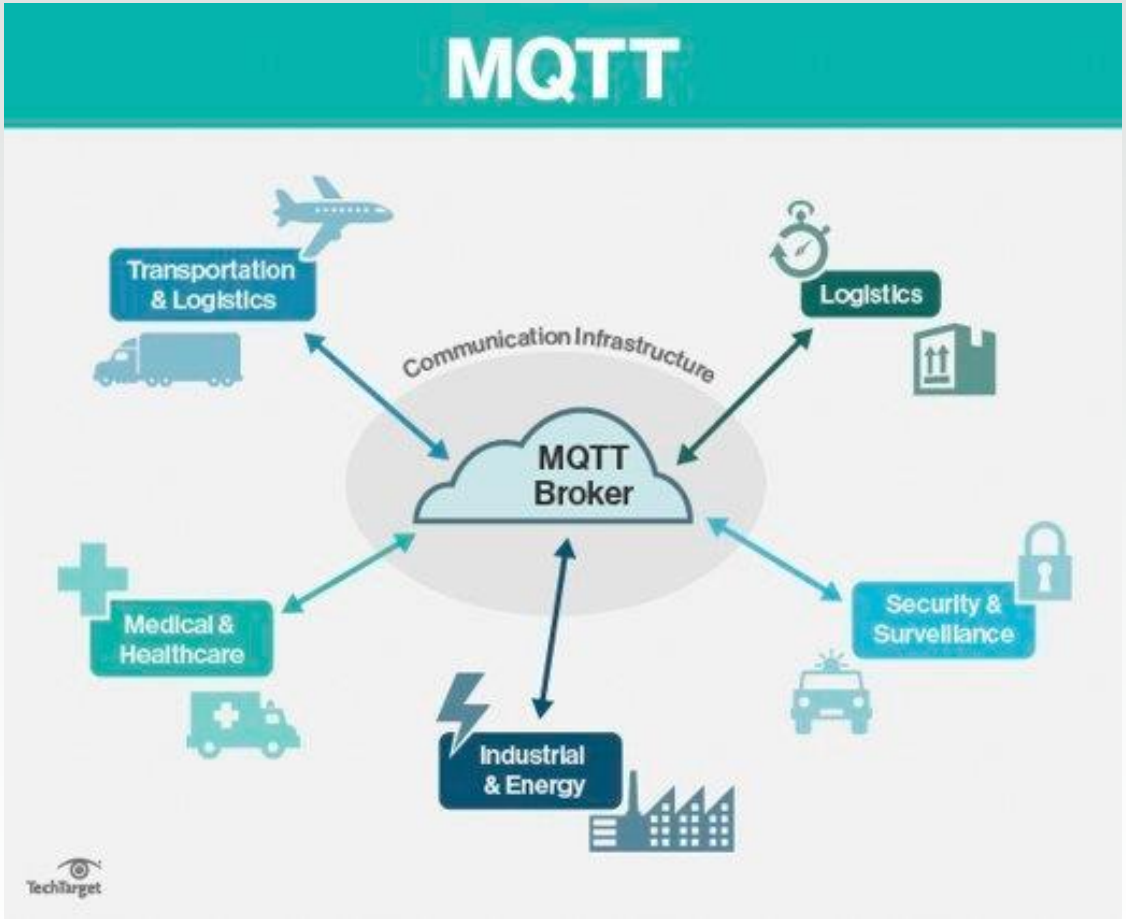
Visit lecture's repository on github: [code]

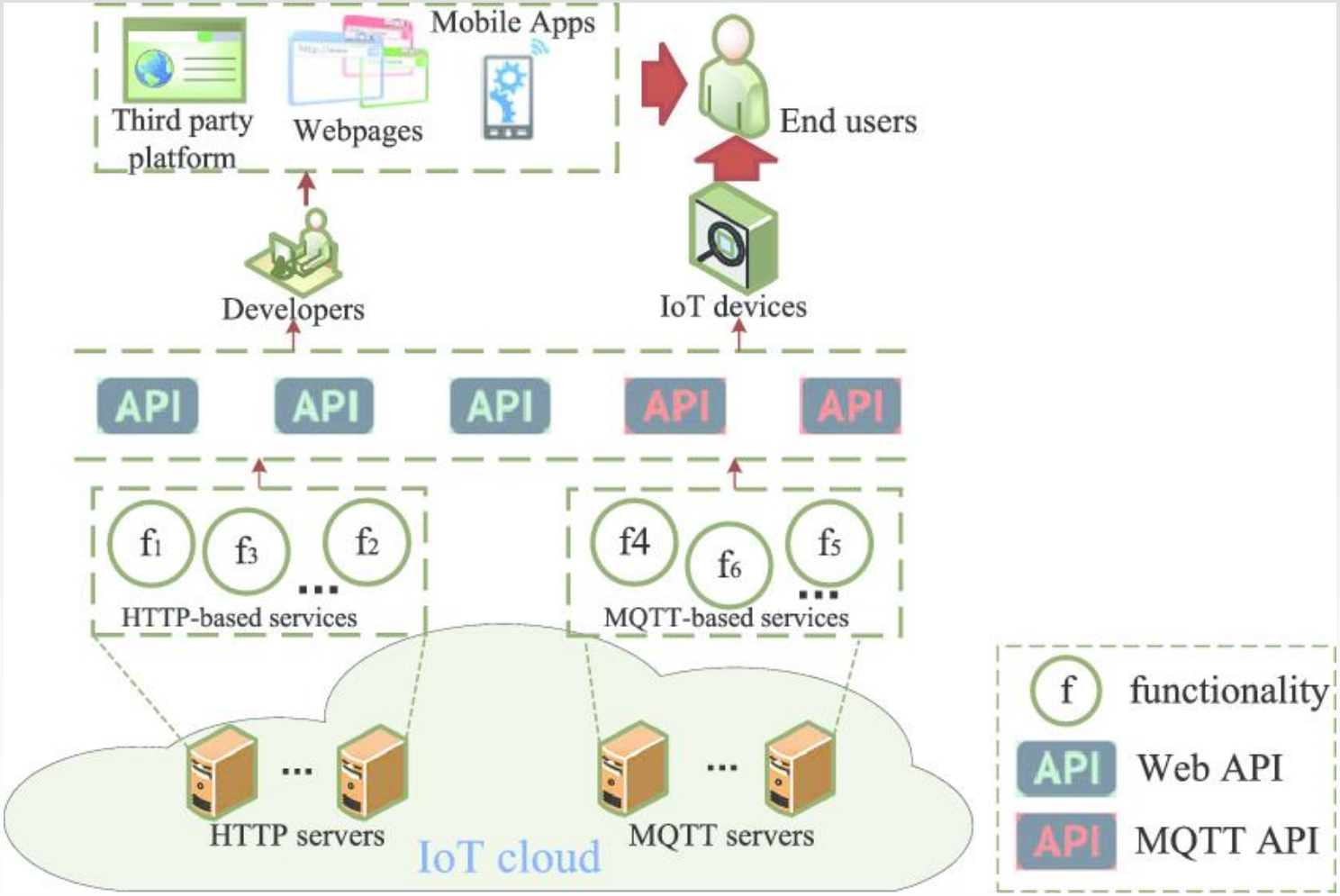
MQTT

Messaging Queuing Telemetry Transport

MQTT Introduction

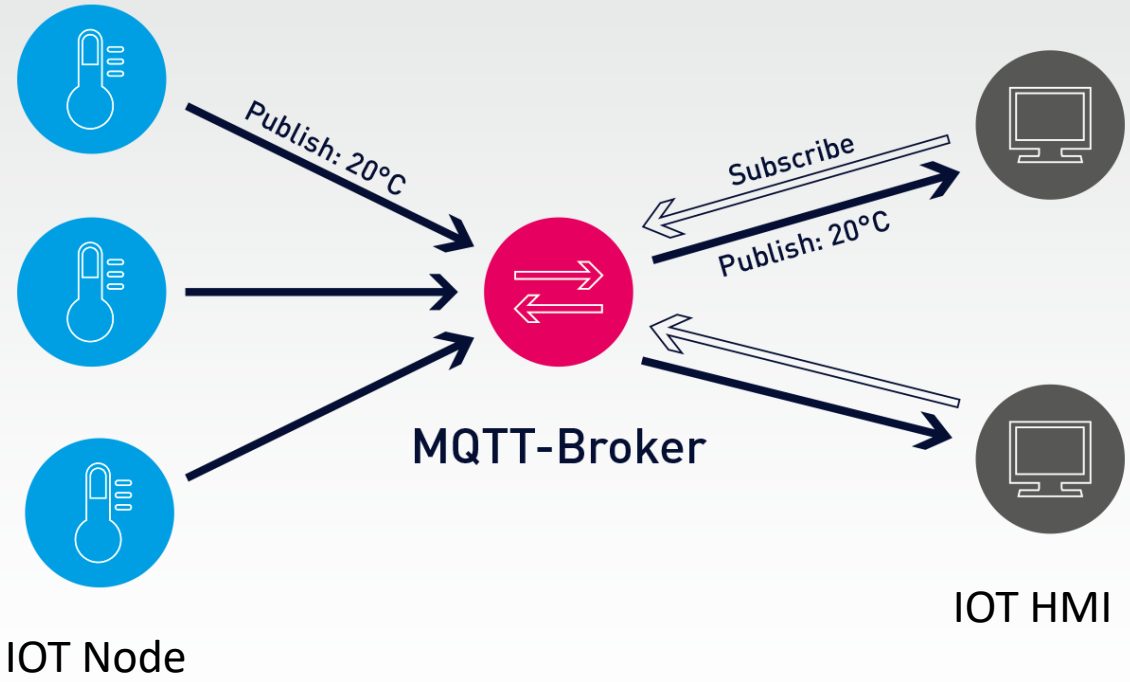
MQTT is a lightweight, publish-subscribe, machine to machine network protocol for message queue/message queuing service. It is designed for connections with remote locations that have devices with resource constraints or limited network bandwidth.



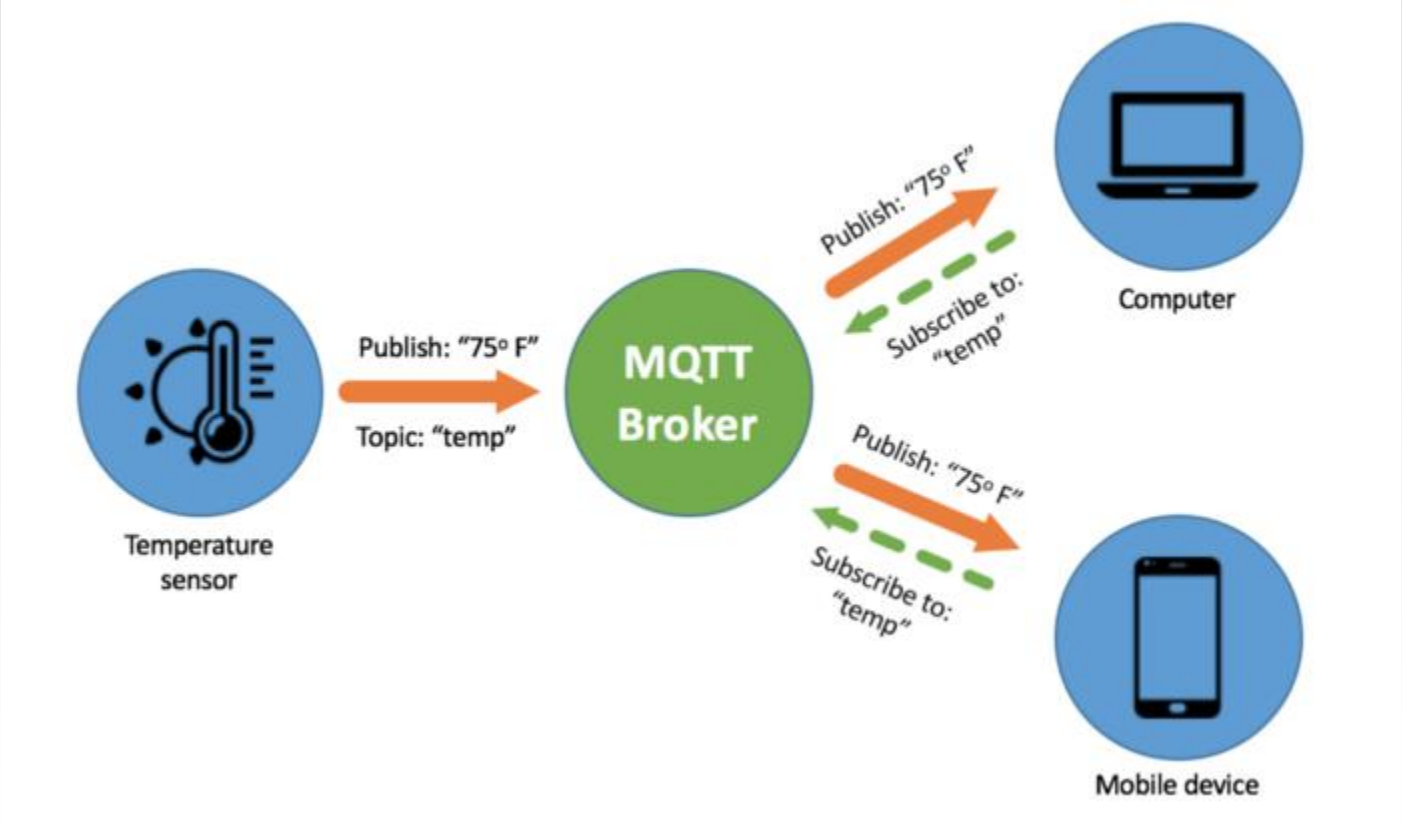

Mobile Apps
Third party platform
Webpages
End users
Developers
IoT devices
API
API
API
API
API
f1
f3
f2
HTTP-based services
f4
f6
f5
MQTT-based services
HTTP servers
IoT cloud
MQTT servers
f functionality
API Web API
API MQTT API

Application Layer MQTT

Message Queuing Telemetry Transport. A lightweight messaging protocol dengan model Publish/Subscribe



MQTT (publisher/subscriber pattern)



Setup a MQTT Broker on NodeJS

Using Node Package Manager

npm website

mqtt-explorer

Get:

- MQTT Broker: Aedes
- MQTT Client: mqtt

port:

- MQTT Broker: 1883
- MQTT Client: 1880

Examples

app.js

```
const http = require("http");
const mqtt = require('mqtt');
const fs = require("fs");
const host = 'localhost';
const port = 8000;

const broker_address = 'mqtt://localhost:1883'

const client = mqtt.connect(broker_address);

var i = 0;

client.on('connect',function(){
    console.log("Publishing to $s",broker_address);

    setInterval(function(){
        i++;
        let message = `hello-hello ${i}`;
        client.publish('test', message);
    },100)
});
```

broker.js

```
const aedes = require('aedes')()
const broker =
require('net').createServer(aedes.handle
)
const broker_port = 1883

broker.listen(broker_port, function () {
  console.log('MQTT broker started and
listening on port ', broker_port)
})
```

Steps

- Open terminal, then run broker
  - node broker.js
  - Don’t close
- Open new terminal, then run app
  - node app.js

ALL Together

NodeJS, MQTT Broker, MQTT Client over HTML
check code on MQTT_pub_sub

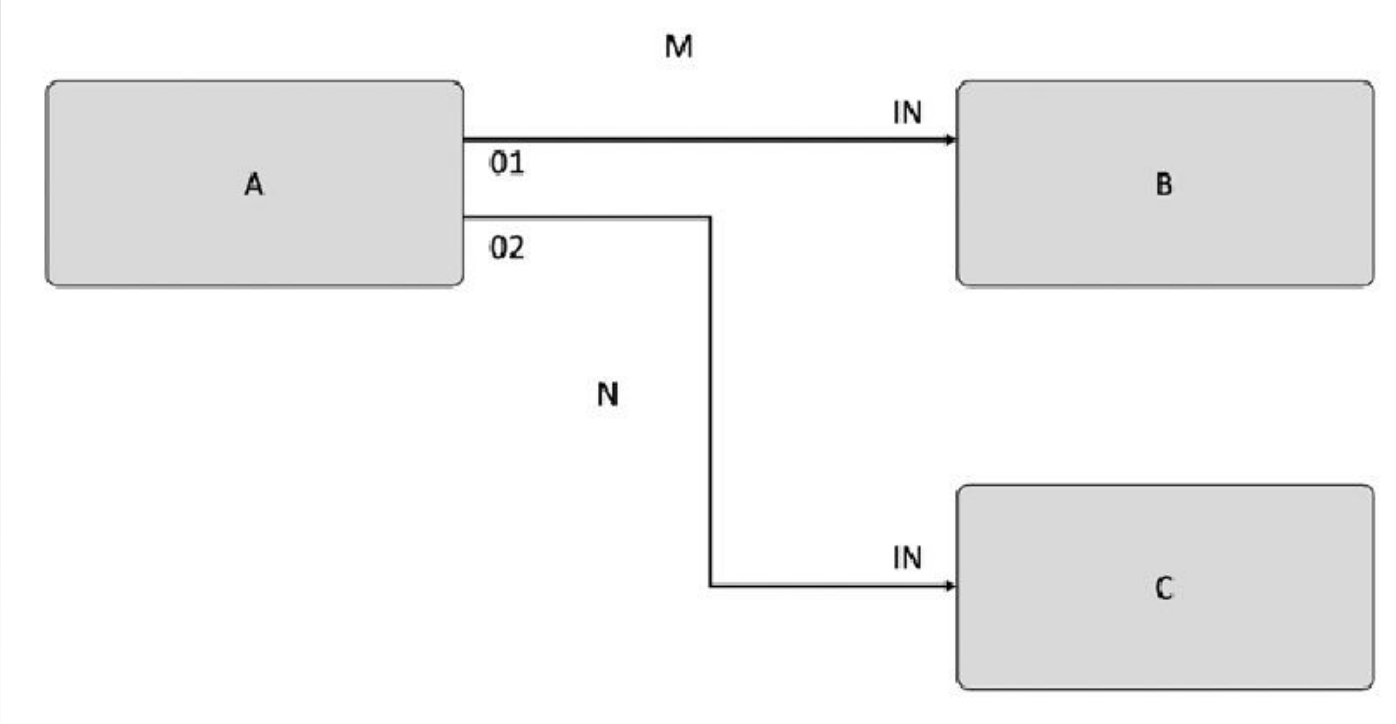
Web Services dengan Node-RED

See installation guide for more detailed steps

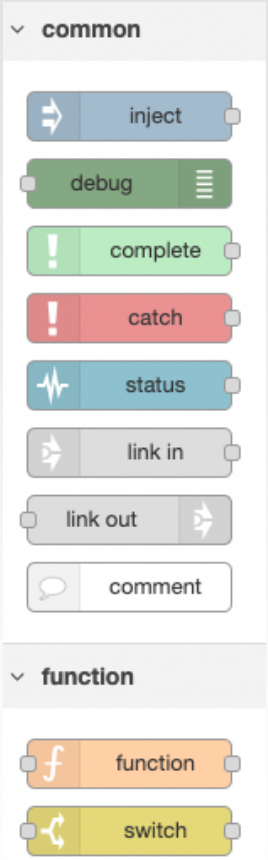
Apa itu NodeRED

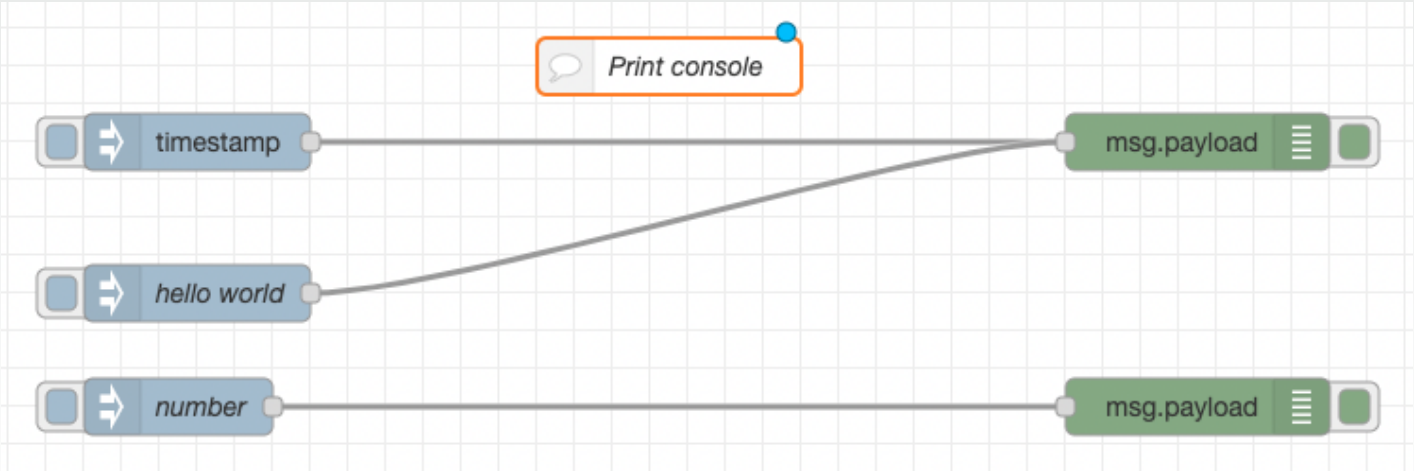
- Node-RED adalah flow-based programming tools yang dikembangkan oleh IBM dan sekarang menjadi bagian dari Javascript Foundation
- Node-RED menggunakan runtime NodeJS
- Node-RED menyediakan editor/tools untuk menghubungkan perangkat IoT (IoT Node, Broker, Client, Database, dll)

Flow Programming

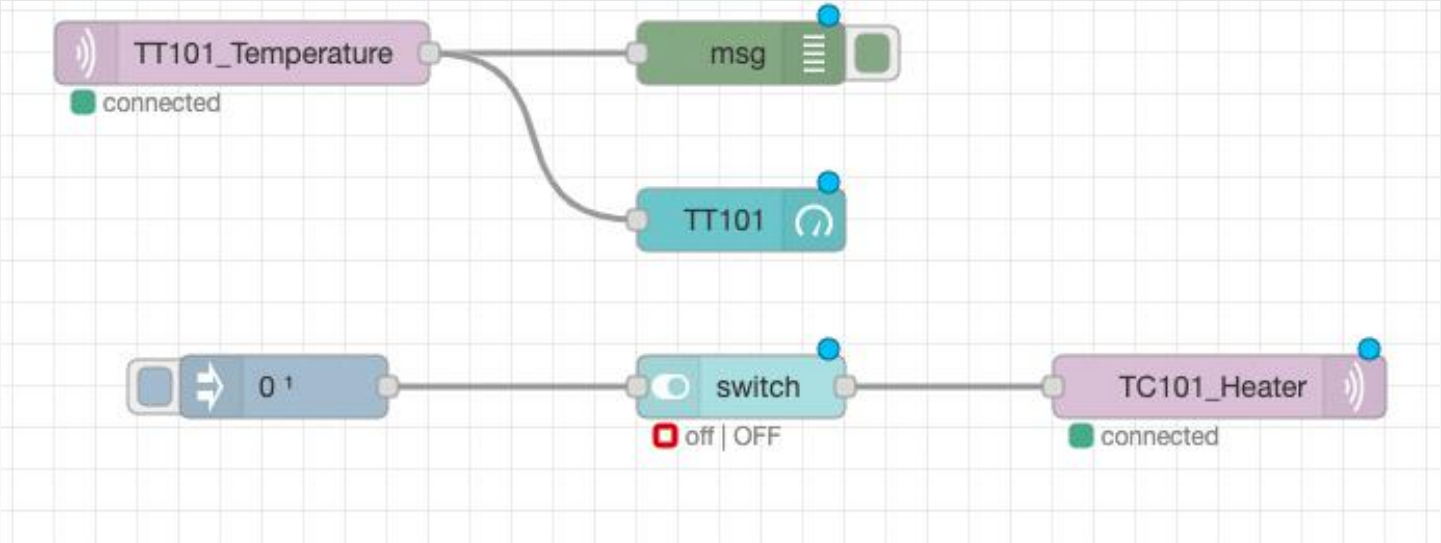


Flow Programming di Node-RED

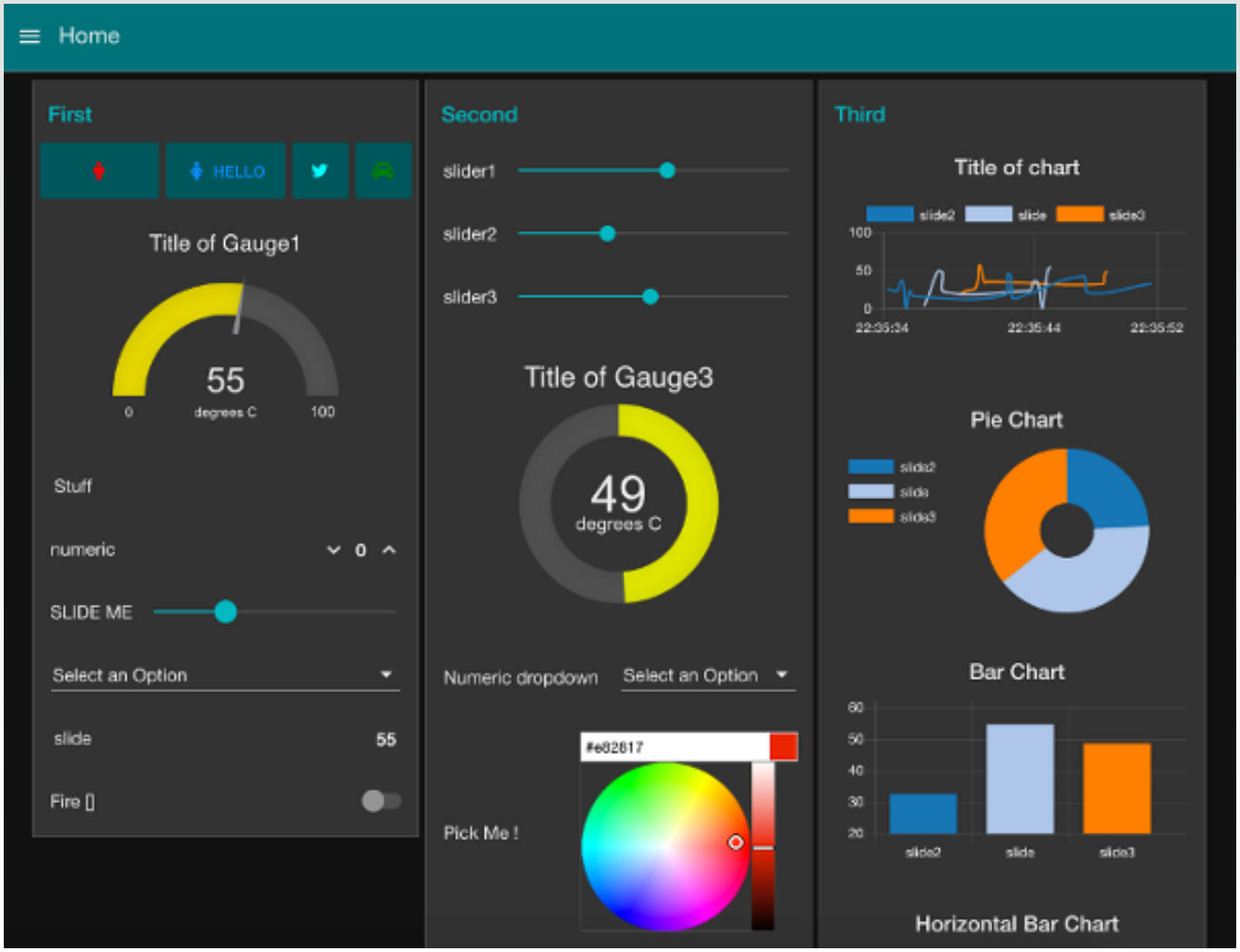




Program IOT Sederhana



Node-Red Dashboard



Node-RED & IoT

- Browser-based UI
- Runtime NodeJS dan Ringan
  - Dapat disematkan pada Edge Devices (ex: Rasp Pi)
  - Dapat dikombinasikan dengan module NodeJS yang lain
- Custom Functions
- External Library
  - Mendukung protocol seperti MQTT dan HTTP